

Cohesity DataProtect

for

PostgreSQL 17/18 on AlmaLinux 8/9

Complete Setup, Configuration, Backup & Restore Guide
Agent Installation | PostgreSQL Connector | CLI & REST API | UI Screenshots

Version 2.0 | March 2026
Subject Matter Expert Reference Document

Table of Contents

1. Introduction and Overview

- 1.1 Architecture Overview
- 1.2 Backup and Restore Workflow

2. Prerequisites and Dependencies

- 2.1 System Packages and Java
- 2.2 PostgreSQL 17/18 Installation
- 2.3 Firewall Configuration

3. Installing the Cohesity Linux Agent

4. Installing the PostgreSQL Connector

5. Configuring PostgreSQL for Backup

- 5.1 WAL Archiving
- 5.2 Create Backup User

6. Registering the PostgreSQL Source

7. Configuring and Running Backups

- 7.1 Create a Protection Policy — Dashboard UI
- 7.2 Create a Protection Group — Dashboard UI
- 7.3 Run Now — On-Demand Backup via Dashboard

8. Command Line Interface (CLI) and REST API

- 8.1 Cohesity iris_cli — Interactive CLI
- 8.2 Cohesity REST API — curl Commands
- 8.3 Cohesity Python SDK
- 8.4 Cohesity PowerShell Module
- 8.5 Shell Script: Automated Backup with Notifications
- 8.6 CLI & API Quick Reference

9. Restore and Recovery Operations

10. Known Limitations and Considerations

- 10.1 Backup Scope Limitations
- 10.2 Version and Compatibility Limitations
- 10.3 High Availability (HA) Limitations
- 10.4 Network, Performance, and Storage Limitations
- 10.5 Point-in-Time Recovery (PITR) Limitations
- 10.6 Operational Limitations
- 10.7 Security and Compliance Limitations
- 10.8 Scalability and Multi-Instance Limitations
- 10.9 Real-World Scenario Examples

11. Troubleshooting Guide

- 11.1 Agent Installation and Connectivity Issues
- 11.2 Source Registration Failures
- 11.3 Backup Failures
- 11.4 Restore Failures
- 11.5 SELinux Troubleshooting
- 11.6 General Diagnostic Checklist
- 11.7 Performance and Timeout Issues
- 11.8 WAL Archive Management Issues
- 11.9 Multi-Node and HA Cluster Issues
- 11.10 Agent Upgrade and Migration Issues
- 11.11 Escalation Procedures and Log Collection

12. Quick Reference Checklist

1. Introduction and Overview

Cohesity DataProtect provides enterprise-grade backup and recovery for PostgreSQL deployments through its Universal Data Adapter (UDA) framework. This guide covers complete setup on AlmaLinux 8/9 with PostgreSQL 17/18, including CLI automation, REST API, and Dashboard UI operations.

1.1 Architecture Overview

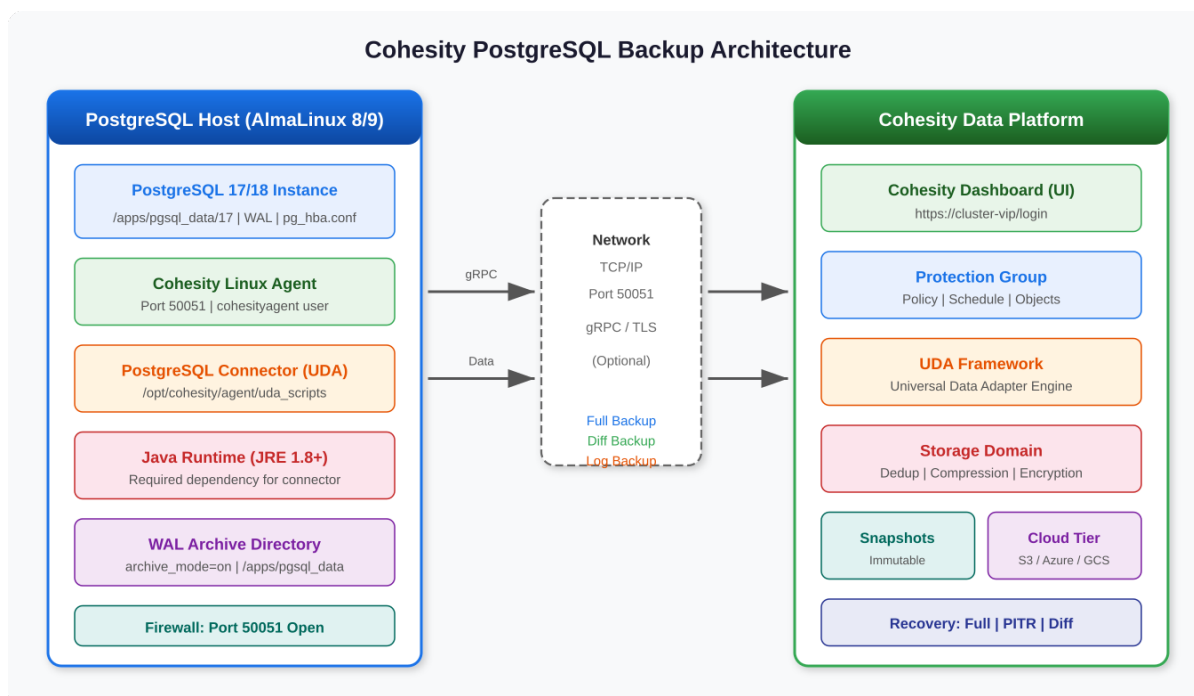


Figure 1: Cohesity PostgreSQL Backup Architecture

1.2 Backup and Restore Workflow

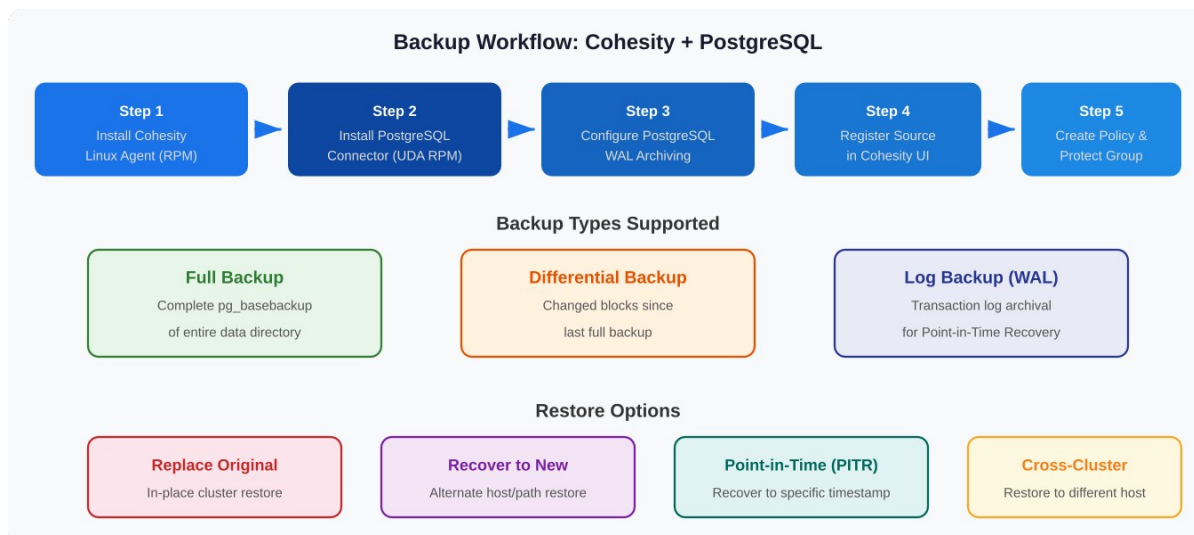


Figure 2: Backup Workflow and Restore Options

2. Prerequisites and Dependencies

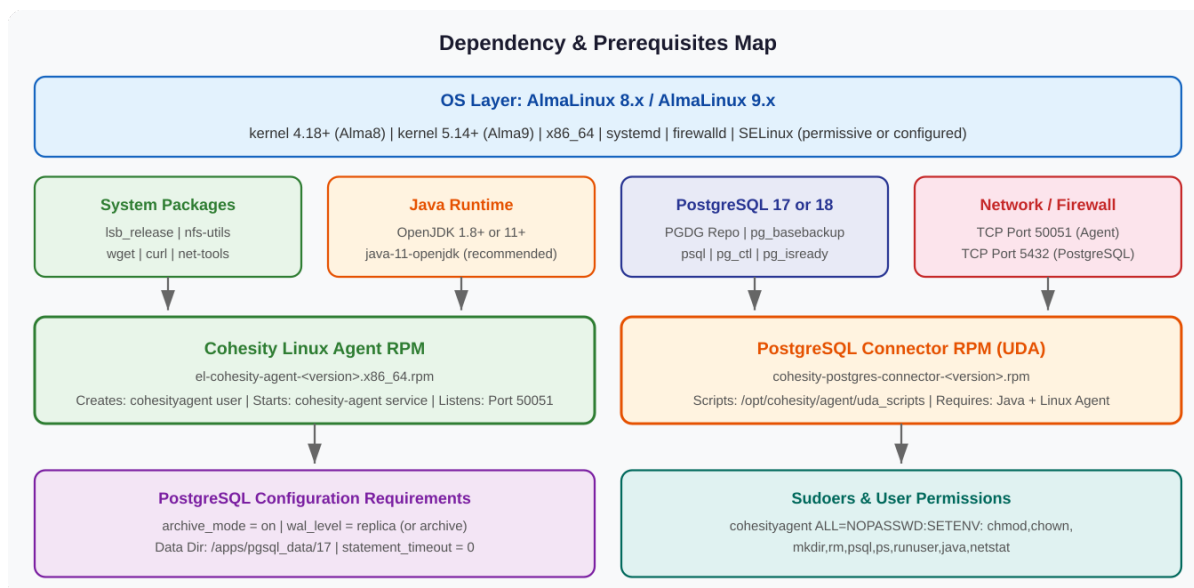


Figure 3: Dependency and Prerequisites Map

2.1 System Packages and Java

```
# AlmaLinux 8/9 - Install required packages
sudo dnf install -y epel-release
sudo dnf install -y wget curl net-tools nfs-utils \
    java-11-openjdk java-11-openjdk-devel

# Verify Java
java -version
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
echo "export JAVA_HOME=$JAVA_HOME" >> /etc/profile.d/java.sh
```

2.2 PostgreSQL 17/18 Installation

```
# Install PGDG repo
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/\
    repopms/EL-$(rpm -E %{rhel})-x86_64/\
    pgdg-redhat-repo-latest.noarch.rpm
sudo dnf -qy module disable postgresql # Alma 8 only

# Install PostgreSQL 17 (or 18)
sudo dnf install -y postgresql17-server postgresql17-contrib

# Create custom data directory
sudo mkdir -p /apps/pgsql_data/17
sudo chown postgres:postgres /apps/pgsql_data/17
```

```
sudo chmod 700 /apps/pgsql_data/17

# Initialize with custom data directory
sudo -u postgres /usr/pgsql-17/bin/initdb \
    -D /apps/pgsql_data/17

# For PostgreSQL 16:
# sudo mkdir -p /apps/pgsql_data/16
# sudo chown postgres:postgres /apps/pgsql_data/16
# sudo -u postgres /usr/pgsql-16/bin/initdb \
#     -D /apps/pgsql_data/16

# Update systemd service to use custom data dir
sudo systemctl edit postgresql-17
# Add the following override:
# [Service]
# Environment=PGDATA=/apps/pgsql_data/17

# Enable and start
sudo systemctl enable postgresql-17 --now
sudo systemctl status postgresql-17
```

2.3 Firewall Configuration

```
sudo firewall-cmd --permanent --add-port=50051/tcp
sudo firewall-cmd --permanent --add-port=5432/tcp
sudo firewall-cmd --reload
```

3. Installing the Cohesity Linux Agent

3.1 Download and Install

Download the Cohesity Linux Agent RPM from the Cohesity Dashboard (Data Protection > Sources > Register > Download Cohesity Agent) and transfer it to the PostgreSQL host.

```
# Install the Cohesity Linux Agent RPM
sudo rpm -ivh el-cohesity-agent-<version>.x86_64.rpm

# Verify installation
rpm -qa | grep cohesity
sudo systemctl status cohesity-agent
id cohesityagent
sudo ss -tlnp | grep 50051
```

3.2 Configure Sudoers

```
sudo visudo

# Add the following entry:
cohesityagent ALL=NOPASSWD:SETENV: /bin/chmod, \
    /bin/chown, /bin/mkdir, /bin/rm, /usr/bin/psql, \
    /usr/bin/ps, /usr/sbin/runuser, /bin/java, \
    /usr/bin/netstat
```


4. Installing the PostgreSQL Connector

Download the PostgreSQL Connector from the Cohesity Download Portal under Datasource Agents.

```
# Install the PostgreSQL Connector RPM
sudo rpm -ivh cohesity-postgres-connector-<version>.rpm
```

```
# Verify scripts directory
ls -la /opt/cohesity/agent/uda_scripts/
```

```
# Set ownership
sudo chown -R cohesityagent:cohesityagent \
/opt/cohesity/agent/uda_scripts/
```

NOTE: Path `/opt/cohesity/agent/uda_scripts/` is required during source registration in the Cohesity Dashboard.

5. Configuring PostgreSQL for Backup

5.1 WAL Archiving

```
# Create the WAL archive directory
sudo mkdir -p /apps/pgsql_data/wal_archive
sudo chown postgres:postgres /apps/pgsql_data/wal_archive
sudo chmod 700 /apps/pgsql_data/wal_archive
```

```
# Edit postgresql.conf (custom data directory):
sudo -u postgres vi /apps/pgsql_data/17/postgresql.conf
```

```
# Add or modify the following parameters:
wal_level = replica
archive_mode = on
archive_timeout = 0
statement_timeout = 0
archive_command = 'test ! -f \
    /apps/pgsql_data/wal_archive/%f \
    && cp %p /apps/pgsql_data/wal_archive/%f'
max_wal_size = 1GB
```

```
# For PG 16, use: /apps/pgsql_data/16/postgresql.conf
```

```
sudo systemctl restart postgresql-17
sudo -u postgres psql -c "SHOW archive_mode;"    # on
sudo -u postgres psql -c "SHOW wal_level;"       # replica
sudo -u postgres psql -c "SHOW data_directory;"
```

```
# Expected: /apps/pgsql_data/17
```

5.2 Create Backup User

```
sudo -u postgres psql
CREATE ROLE cohesity_backup WITH LOGIN SUPERUSER \
    PASSWORD 'StrongPassword123!';
\q
```

6. Registering the PostgreSQL Source

Register the PostgreSQL deployment as a data source in the Cohesity Dashboard using the Universal Data Adapter interface.

6.1 Dashboard UI: Register Source

The screenshot shows the Cohesity Dashboard interface for registering a PostgreSQL source. The browser address bar shows `https://cohesity-cluster.company.com/data-protection/sources/register`. The dashboard has a dark sidebar with 'Cohesity' logo and 'Data Protection' section containing 'Sources', 'Protection', and 'Policies'. The main content area is titled 'Register Universal Data Adapter' and contains the following fields:

- Source Type ***: A dropdown menu with 'PostgreSQL' selected.
- Host OS Type ***: A dropdown menu with 'Linux' selected.
- Hostname / IP Addresses ***: A text input field containing '192.168.1.50'.
- Datasource Agent Installation Path ***: A text input field containing '/opt/cohesity/agent/uda_scripts'.
- Authentication Settings**: A section with a yellow background containing:
 - Username ***: A text input field containing 'cohesity_backup'.
 - Password ***: A password input field with masked characters '*****'.
- SSL Settings (Optional)**: A section with a green background containing two checkboxes:
 - ☐ Enable PostgreSQL Client SSL Setting
 - ☐ Enable Cohesity SSL Settings (Secure gRPC)

At the bottom right of the form are 'Cancel' and 'Register' buttons. Below the form, a small caption reads: 'Figure: Cohesity Dashboard — Register PostgreSQL Source (Universal Data Adapter)'.

Figure 4: Register PostgreSQL Source — Cohesity Dashboard UI

1. Navigate to Data Protection > Sources > Register > Universal Data Adapter.
2. Select Source Type: PostgreSQL, Host OS Type: Linux.
3. Enter the hostname/IP of the PostgreSQL host (e.g., 192.168.1.50).
4. Enter the Datasource Agent Installation Path: /opt/cohesity/agent/uda_scripts
5. In Authentication Settings, enter the backup user credentials (cohesity_backup).
6. Optionally enable PostgreSQL Client SSL and Cohesity SSL (gRPC).
7. Click Register to complete.

7. Configuring and Running Backups

7.1 Create a Protection Policy — Dashboard UI

A Protection Policy defines how and when backups are taken. The following screenshot shows the Create Policy interface:

The screenshot displays the 'Create Protection Policy' form in the Cohesity Dashboard. The form is titled 'Create Protection Policy' and includes a subtitle 'Configure backup schedules, retention, and replication settings for PostgreSQL protection.' The form is divided into several sections: 'Policy Name' and 'Description' at the top, followed by 'Backup Schedule' which includes 'Incremental Backup' (Frequency: Every 1 Day, Start Time: 02:00 AM), 'Periodic Full Backup' (Enabled, Every: 7 Days), and 'Log Backup (WAL)' (Enabled, Every: 15 Minutes). Below these is the 'Retention Settings' section with 'Local Retention' (30 Days), 'Archival Retention' (90 Days), and 'Log Retention' (14 Days). A link for 'More Options (Replication, Archival, Advanced Settings)' is provided. At the bottom right are 'Cancel' and 'Create' buttons.

Figure: Cohesity Dashboard — Create Protection Policy for PostgreSQL

Figure 5: Create Protection Policy — Cohesity Dashboard UI

Recommended Policy Settings:

Setting	Recommended Value
Policy Name	PG17-Daily-Full-Weekly-Policy
Incremental Backup	Every 1 Day at 2:00 AM
Full Backup	Every 7 Days (weekly)
Log Backup (WAL)	Every 15 Minutes
Local Retention	30 Days
Archival Retention	90 Days

7.2 Create a Protection Group — Dashboard UI

Link the PostgreSQL source to the Protection Policy by creating a Protection Group:

The screenshot displays the Cohesity Dashboard UI for creating a Protection Group. The browser address bar shows `https://cohesity-cluster.company.com/data-protection/protection`. The dashboard has a dark blue header with 'Cohesity' and navigation links for 'Dashboard', 'Data Protection', and 'Marketplace'. A left sidebar lists 'DATA PROTECTION', 'Sources', 'Protection', 'Policies', and 'Recoveries'. The main content area is titled 'Protect Universal Data Adapter' with the instruction 'Configure protection for your PostgreSQL database source using Universal Data Adapter.' It features three sections: 'Source Configuration' with a dropdown for 'Source' (pg-alma9-prod01 (192.168.1.50)) and a 'Source Registered & Healthy' status; 'Objects' with an 'Object Name' input (pg17-production-cluster) and a '+ Add more objects' button; and 'Protection Group & Policy' with 'Protection Group Name' (PG17-Alma9-ProtectionGroup) and 'Policy' (PG17-Daily-Full-Weekly-Policy). A 'Policy Summary' box shows 'Incremental: Daily 2AM | Full: Weekly' and 'Logs: Every 15min | Retain: 30 days'. At the bottom are 'Cancel' and 'Protect' buttons.

Figure 6: Cohesity Dashboard — Create Protection Group for PostgreSQL (UDA)

Figure 6: Create Protection Group — Cohesity Dashboard UI

1. Navigate to Data Protection > Protection > Protect > Universal Data Adapter.
2. Select the registered PostgreSQL source.
3. Enter an Object Name for tracking (e.g., pg17-production-cluster).
4. Enter a Protection Group Name (e.g., PG17-Alma9-ProtectionGroup).
5. Select the Policy created in step 7.1 from the drop-down.
6. Review Additional Settings (concurrency, mounts, pre/post scripts).
7. Click Protect to activate.

7.3 Run Now — On-Demand Backup via Dashboard

To trigger an immediate on-demand backup, use the actions menu on the Protection Group:

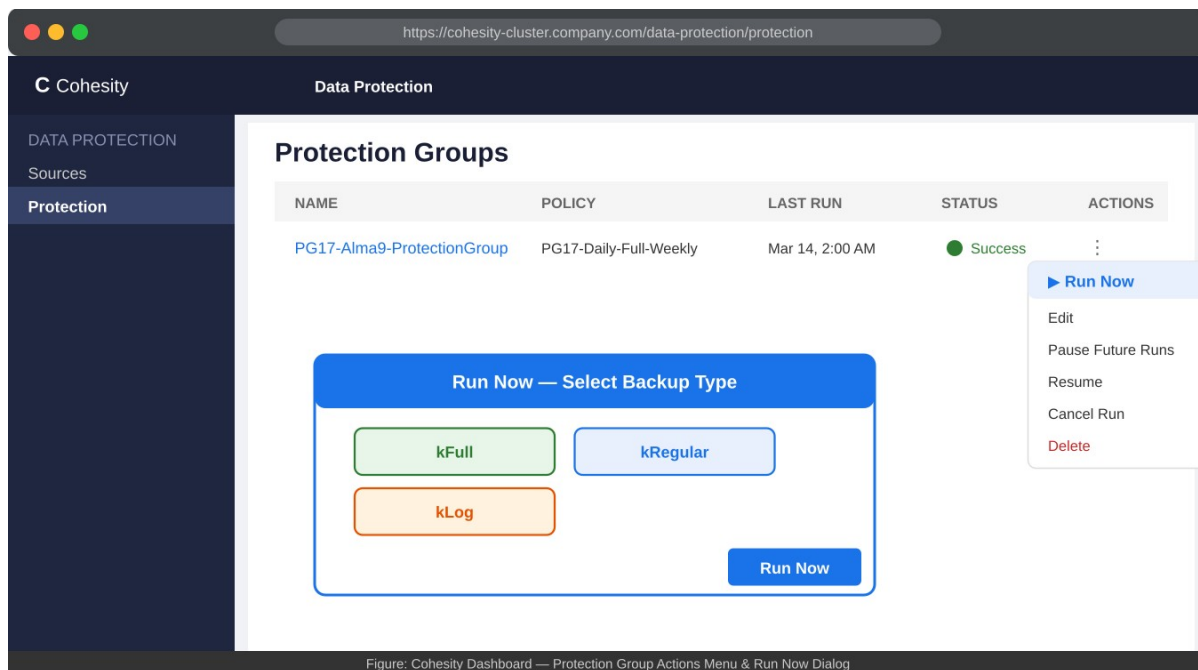


Figure: Cohesity Dashboard — Protection Group Actions Menu & Run Now Dialog

Figure 7: Protection Group — Run Now Dialog

1. Navigate to Data Protection > Protection.
2. Click the three-dot actions menu (⋮) next to the Protection Group.
3. Select Run Now.
4. Choose the backup type: kFull (complete), kRegular (incremental/differential), or kLog (WAL logs).
5. Click Run Now to start the backup immediately.

8. Command Line Interface (CLI) and REST API

Cohesity provides multiple command-line and programmatic methods to manage backups beyond the Dashboard UI. This section covers the `iris_cli` (Cohesity CLI), REST API with `curl`, the Python SDK, and the PowerShell module.

8.1 Cohesity `iris_cli` — Interactive CLI

The `iris_cli` is Cohesity's native command-line interface, accessible by SSH-ing into a Cohesity cluster node. It provides cluster management and operational commands.

8.1.1 Connecting to `iris_cli`

```
# SSH into the Cohesity cluster node
ssh support@<cohesity-node-ip>

# Launch the Cohesity CLI
iris_cli
Username: admin
Password: *****

# Or use non-interactive mode:
iris_cli -server <cluster-vip> \
    -username admin -password <password> \
    -domain LOCAL
```

8.1.2 Common `iris_cli` Commands

Command	Description
cluster status	Display cluster health, services status, and software version
cluster info	Show cluster details including ID, name, and incarnation
cluster ls-nodes	List all nodes in the Cohesity cluster
protection-job list	List all protection jobs/groups on the cluster
protection-job run <job-name>	Trigger an on-demand backup run for a protection job
protection-job pause <job-name>	Pause future scheduled runs of a protection job
protection-job resume <job-name>	Resume a paused protection job
user add user-name=<> password=<>	Create a new Cohesity cluster user
cluster update-gflag ...	Update global flags for cluster services

TIP: Use 'help' at any `iris_cli` prompt to see all available commands. Use '<command> help' to see operations for a specific command.

8.2 Cohesity REST API — curl Commands

The Cohesity REST API provides full programmatic control over backup operations. All API calls require an access token obtained by authenticating against the cluster.

8.2.1 Authenticate and Get Access Token

```
# Authenticate and retrieve access token
CLUSTER_VIP="cohesity-cluster.company.com"

TOKEN=$(curl -sk -X POST \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/accessTokens" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  -d '{
    "domain": "LOCAL",
    "username": "admin",
    "password": "YourPassword"
  }' | python3 -c \
  "import sys,json;print(json.load(\
sys.stdin)['accessToken'])")

echo "Token: ${TOKEN}"
```

8.2.2 List Protection Groups

```
# List all protection groups
curl -sk -X GET \
  "https://${CLUSTER_VIP}/v2/data-protect/\
protection-groups" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Accept: application/json' | python3 -m json.tool
```

8.2.3 Run an On-Demand Backup via REST API

```
# Trigger an on-demand backup (Full)
# Replace <JOB_ID> with the actual protection group ID

curl -sk -X POST \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/protectionJobs/run/<JOB_ID>" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  -d '{
    "runType": "kFull"
  }'
```



```
}'

# For Incremental (CBT) backup:
# "runType": "kRegular"
```

```
# For Log (WAL) backup:
# "runType": "kLog"
```

NOTE: Available runType values: 'kRegular' (incremental/CBT), 'kFull' (complete backup, no CBT), 'kLog' (database transaction logs/WAL), 'kSystem' (system/bare-metal backup).

8.2.4 Check Backup Run Status

```
# Check protection runs for a specific job
curl -sk -X GET \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/protectionRuns?jobId=<JOB_ID>\
&numRuns=5" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Accept: application/json' | python3 -m json.tool
```

8.2.5 List Registered Sources

```
# List all registered protection sources
curl -sk -X GET \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/protectionSources" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Accept: application/json' | python3 -m json.tool
```

8.2.6 List Protection Policies

```
# List all protection policies
curl -sk -X GET \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/protectionPolicies" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Accept: application/json' | python3 -m json.tool
```

8.2.7 Create a Protection Policy via REST API

```
# Create a new protection policy
curl -sk -X POST \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/protectionPolicies" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
```

```
"name": "PG17-API-Policy",
"incrementalSchedulingPolicy": {
  "periodicity": "kDaily",
  "dailySchedule": {
    "days": ["kMonday", "kTuesday",
             "kWednesday", "kThursday",
             "kFriday", "kSaturday", "kSunday"]
  }
},
"daysToKeep": 30,
"retries": 3,
"retryIntervalMins": 30
}'
```

8.3 Cohesity Python SDK

The `cohesity_management_sdk` Python package provides a programmatic interface for all Cohesity operations including backup job management.

8.3.1 Installation

```
# Install from PyPI
pip install cohesity_management_sdk

# Or install from source:
git clone https://github.com/cohesity/management-sdk-python.git
cd management-sdk-python
pip install -r requirements.txt
python setup.py install
```

8.3.2 Initialize the Client

```
from cohesity_management_sdk.cohesity_client \
    import CohesityClient

# Connect to the Cohesity cluster
client = CohesityClient(
    cluster_vip='cohesity-cluster.company.com',
    username='admin',
    password='YourPassword',
    domain='LOCAL'
)

# Verify connection
cluster_ctrl = client.cluster
info = cluster_ctrl.get_basic_cluster_info()
print(f"Connected to: "
      f"{info.cluster_software_version}")
```

8.3.3 List Protection Jobs

```
# List all protection jobs
jobs_ctrl = client.protection_jobs
jobs = jobs_ctrl.get_protection_jobs()

for job in jobs:
    print(f"Job: {job.name} "
          f"| ID: {job.id} "
          f"| Env: {job.environment}")
```

8.3.4 Trigger an On-Demand Backup

```
from cohesity_management_sdk.models\
    .run_protection_job_param \
import RunProtectionJobParam

# Trigger a full backup for job ID 12345
job_id = 12345
run_param = RunProtectionJobParam()
run_param.run_type = 'kFull' # or kRegular, kLog

jobs_ctrl = client.protection_jobs
jobs_ctrl.create_run_protection_job(
    id=job_id,
    body=run_param
)
print(f"Backup triggered for job {job_id}")
```

8.3.5 Monitor Backup Runs

```
# Get recent protection runs
runs_ctrl = client.protection_runs
runs = runs_ctrl.get_protection_runs(
    job_id=12345,
    num_runs=5
)

for run in runs:
    status = run.backup_run.status
    start = run.backup_run.stats.start_time_usecs
    print(f"Run: {start} | Status: {status}")
```

8.4 Cohesity PowerShell Module

For Windows administrators, Cohesity provides a PowerShell module for automation.

8.4.1 Installation and Connection

```
# Install the Cohesity PowerShell module
Install-Module -Name Cohesity.PowerShell

# Connect to the cluster
Connect-CohesityCluster \
    -Server cohesity-cluster.company.com \
    -Credential (Get-Credential)
```

8.4.2 Backup Operations

```
# List all protection jobs
Get-CohesityProtectionJob

# Get a specific protection job
Get-CohesityProtectionJob \
  -Names 'PG17-Alma9-ProtectionGroup'

# Trigger an on-demand full backup
Start-CohesityProtectionJob \
  -Name 'PG17-Alma9-ProtectionGroup' \
  -RunType kFull

# Trigger an incremental backup
Start-CohesityProtectionJob \
  -Name 'PG17-Alma9-ProtectionGroup' \
  -RunType kRegular

# Trigger a log (WAL) backup
Start-CohesityProtectionJob \
  -Name 'PG17-Alma9-ProtectionGroup' \
  -RunType kLog

# Check recent backup run status
Get-CohesityProtectionJobRun \
  -JobName 'PG17-Alma9-ProtectionGroup' \
  -NumRuns 5

# Pause a protection job
Suspend-CohesityProtectionJob \
  -Name 'PG17-Alma9-ProtectionGroup'

# Resume a paused protection job
Resume-CohesityProtectionJob \
  -Name 'PG17-Alma9-ProtectionGroup'
```

8.5 Shell Script: Automated Backup with Notifications

The following shell script combines the REST API with standard Linux tools to automate backup runs with email notifications:

```
#!/bin/bash
# cohesity_pg_backup.sh - Automated PG backup
# Usage: ./cohesity_pg_backup.sh [kFull|kRegular|kLog]
```

```

CLUSTER="cohesity-cluster.company.com"
USER="admin"
PASS="YourPassword"
JOB_ID="12345"
RUN_TYPE=${1:-kRegular}
LOG="/var/log/cohesity_backup.log"

# Get access token
TOKEN=$(curl -sk -X POST \
  "https://${CLUSTER}/irisservices/api/v1/\
public/accessTokens" \
  -H 'Content-Type: application/json' \
  -d '{"domain":"'LOCAL'",\
  "username":"'${USER}'",\
  "password":"'${PASS}'"}' \
  | python3 -c "import sys,json;\
print(json.load(sys.stdin)['accessToken'])")

# Trigger backup
RESPONSE=$(curl -sk -X POST \
  "https://${CLUSTER}/irisservices/api/v1/\
public/protectionJobs/run/${JOB_ID}" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{"runType":"'${RUN_TYPE}'"}' \
  -w "%{http_code}" -o /dev/null)

TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
if [ "$RESPONSE" = "204" ]; then
  echo "[${TIMESTAMP}] SUCCESS: ${RUN_TYPE} \
backup triggered" >> $LOG
else
  echo "[${TIMESTAMP}] FAILED: HTTP \
${RESPONSE}" >> $LOG
fi

```

Schedule with cron:

```

# Daily incremental at 2:00 AM
0 2 * * * /opt/scripts/cohesity_pg_backup.sh kRegular

# Weekly full on Sunday at 1:00 AM
0 1 * * 0 /opt/scripts/cohesity_pg_backup.sh kFull

```

```
# WAL log backup every 15 minutes
*/15 * * * * /opt/scripts/cohesity_pg_backup.sh kLog
```

NOTE: While cron-based scheduling is possible, the recommended approach is to use Cohesity Protection Policies for scheduling, as they provide built-in retry logic, SLA tracking, and centralized management.

8.6 CLI & API Quick Reference

Method	Trigger Full Backup	Check Status
Dashboard UI	Protection > ⋮ > Run Now > kFull	Protection > Click Group > View Runs
iris_cli	protection-job run <job-name>	protection-job list
REST API	POST /protectionJobs/run/<ID> { "runType": "kFull" }	GET /protectionRuns?jobId=<ID>
Python SDK	create_run_protection_job(id, RunProtectionJobParam(run_type='kFull'))	get_protection_runs(job_id, num_runs=5)
PowerShell	Start-CohesityProtectionJob -Name <> -RunType kFull	Get-CohesityProtectionJobRun - JobName <>
cron + curl	0 1 * * 0 /opt/scripts/cohesity_pg_backup.sh kFull	Check /var/log/cohesity_backup.log

9. Restore and Recovery Operations

9.1 Restore via Dashboard UI

1. Navigate to Data Protection > Recoveries > Recover > Universal Data Adapter.
2. Select the PostgreSQL source and specify an Object Name.
3. Select the Recovery Point (latest snapshot, specific snapshot, or point-in-time).
4. Click Next: Recovery Options.
5. Choose Replace Original or Recover to a New Object.
6. For Recover to New: set --target-dir=/apps/pgsql_data/17_restored in Recovery Script Arguments.
7. Click Recover and monitor progress.

9.2 Restore via REST API

```
# Initiate a restore via REST API
curl -sk -X POST \
  "https://${CLUSTER_VIP}/irisservices/api/v1/\
public/restore/recover" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "PG17-Restore-$(date +%Y%m%d)",
    "type": "kRecoverApp",
    "action": "kRecoverApp",
    "restoreAppParams": {
      "type": 24,
      "ownerRestoreInfo": {
        "ownerObject": {
          "jobId": <JOB_ID>,
          "jobUid": { ... },
          "jobInstanceId": <RUN_ID>
        }
      }
    }
  }'
```

NOTE: Restore API payloads are complex and job-specific. It is recommended to use the Dashboard UI for most restore operations or export the payload from the Cohesity API documentation for your cluster version.

9.3 Post-Restore Steps

```
# For Replace Original:
sudo systemctl start postgresql-17
sudo -u postgres pg_isready
```

```
# For Recover to New Object:
sudo chown -R postgres:postgres \
    /apps/pgsql_data/17_restored
sudo -u postgres /usr/pgsql-17/bin/pg_ctl start \
    -D /apps/pgsql_data/17_restored -o '-p 5433'

# Verify restored instance
psql -h localhost -p 5433 -U postgres \
    -c "SELECT version();"

# Re-enable systemd tracking after restore:
sudo -u postgres /usr/pgsql-17/bin/pg_ctl stop \
    -D /apps/pgsql_data/17
sudo systemctl start postgresql-17
```

10. Known Limitations and Considerations

Before deploying Cohesity DataProtect for PostgreSQL in production, it is critical to understand the following limitations, known issues, and design considerations. Failure to account for these can result in unexpected backup failures, incomplete restores, or unmet SLA requirements.

10.1 Backup Scope Limitations

#	Limitation	Detail / Impact / Workaround
1	Physical backup only	Cohesity uses <code>pg_basebackup</code> under the hood. Logical backups (<code>pg_dump</code> , <code>pg_dumpall</code>) are NOT managed by Cohesity. If you need logical exports for specific databases, schemas, or tables, you must run <code>pg_dump</code> separately and manage those dumps outside of Cohesity.
2	Cluster-level backup/restore only	Backup and restore operate on the entire PostgreSQL data directory. You CANNOT selectively backup or restore a single database, schema, or table. The entire cluster (all databases within the instance) is always included. For object-level granularity, use <code>pg_dump</code> alongside Cohesity.
3	No tablespace-level backup	Individual tablespaces cannot be backed up or restored independently. All tablespaces associated with the PostgreSQL instance are included in the full backup. Custom tablespace locations must be accessible during restore.
4	No streaming replication slot backup	Replication slots and their state are not preserved across backup/restore. After restoring, you must manually recreate replication slots and re-establish streaming replication to standby servers.

10.2 Version and Compatibility Limitations

#	Limitation	Detail / Impact / Workaround
5	Cross-version restore NOT supported	A backup taken from PostgreSQL 17 CANNOT be restored to PostgreSQL 18, and vice versa. The source and target must run the exact same major version. This also means you cannot use Cohesity backups as a migration path for major version upgrades. Use <code>pg_upgrade</code> or <code>pg_dump/pg_restore</code> for cross-version migrations.
6	Cohesity agent version compatibility	The Cohesity Linux Agent and PostgreSQL Connector versions must be compatible with the Cohesity cluster version. Always check the Cohesity compatibility matrix before upgrading any component. Mismatched versions may cause backup failures or registration errors.
7	PostgreSQL version support	Only officially supported PostgreSQL versions are guaranteed to work. Check the Cohesity 'Databases' compatibility page for your cluster version. Community PostgreSQL and enterprise distributions (EDB, Aurora) may

		have different support levels.
--	--	--------------------------------

10.3 High Availability (HA) Limitations

#	Limitation	Detail / Impact / Workaround
8	Backup runs on active node only	In an HA cluster (Patroni, repmgr, pgpool), Cohesity executes the backup only from the active/primary node. Standby nodes are NOT used for backups. If the standby is promoted, the agent must be installed and registered on that node as well.
9	Failover triggers full backup	In the event of an HA failover, the next backup run will automatically be a FULL backup regardless of the policy schedule. This is because differential/incremental tracking is reset after failover. Plan storage capacity accordingly for environments with frequent failovers.
10	No automatic failover-aware registration	Cohesity does not automatically detect HA failovers or re-point to the new primary. If using a VIP (virtual IP) that floats with the primary, register using the VIP. Otherwise, you may need to manually update the source registration after a failover event.

10.4 Network, Performance, and Storage Limitations

#	Limitation	Detail / Impact / Workaround
11	IPv6-only NOT supported	The PostgreSQL server must listen on an IPv4 address. IPv6-only deployments are not yet supported. Dual-stack (IPv4+IPv6) mode IS supported. Ensure listen_addresses in postgresql.conf includes an IPv4 address.
12	Secure gRPC increases transfer time	Enabling TLS encryption for gRPC communication between the Cohesity cluster and the PostgreSQL host increases data transfer overhead. For large databases (multi-TB), this can significantly extend backup windows. Test performance impact before enabling in production.
13	WAL segment size recommendation	For optimum log backup performance, Cohesity recommends setting max_wal_size = 1GB. Smaller WAL segment sizes may lead to excessive archiving overhead and slower log backups.
14	Port conflict on restore	When restoring to the same host, ensure the target port for the restored instance is NOT already in use. If the original PostgreSQL instance is still running on port 5432, restore to an alternate port (e.g., 5433) or stop the original instance first.

10.5 Point-in-Time Recovery (PITR) Limitations

#	Limitation	Detail / Impact / Workaround
15	PITR gap after backup timeline holes	If a hole (missed backup) is introduced in the backup timeline — for example, due to a paused protection group, agent downtime, or network failure — PITR is only possible AFTER the next successful full or incremental backup completes. WAL logs from before the gap cannot bridge across it.
16	archive_mode=on restarts the database	If archive_mode is NOT already set to 'on' when the first backup runs, Cohesity will automatically modify postgresql.conf and RESTART the PostgreSQL service. This causes a brief outage. Always pre-configure archive_mode=on before registering the source to avoid unexpected downtime.
17	PITR granularity depends on log backup frequency	The granularity of PITR is directly tied to the log backup frequency configured in the Protection Policy. With 15-minute log backups, you can recover to any point within the last 15 minutes of WAL. More frequent log backups (e.g., every 5 minutes) improve RPO but increase storage consumption.

10.6 Operational Limitations

#	Limitation	Detail / Impact / Workaround
18	Restore does not use systemd	After a restore, Cohesity restarts PostgreSQL using sudo/runuser/pg_ctl directly, NOT via systemctl. This means systemd does not track the restored process. To re-enable systemd tracking, stop the database with pg_ctl and restart it using systemctl.
19	No parallel object-level restore	Since only cluster-level restore is supported, you cannot restore multiple individual databases in parallel from the same snapshot. The entire data directory is restored as one unit.
20	Sudoers management with config tools	If your server's /etc/sudoers file is managed by a deployment engine (Chef, Puppet, Ansible), the engine may overwrite the cohesityagent sudo entries on its next run. Ensure the Cohesity sudoers entries are included in your configuration management templates.
21	No encryption-at-rest key management	If PostgreSQL uses data-at-rest encryption (e.g., TDE via extensions), the encryption keys must be managed separately. Cohesity backs up the encrypted data directory but does NOT manage or back up the encryption keys. Ensure key backup is handled independently.

10.7 Security and Compliance Limitations

#	Limitation	Detail / Impact / Workaround
22	Backup credentials stored in Cohesity	The PostgreSQL backup user's username and password are stored in the Cohesity cluster as part of the source registration. Ensure your Cohesity cluster uses encryption at rest and that RBAC (Role-Based Access Control) limits who can view registered sources. Rotate the backup user password periodically and update the source registration accordingly.
23	No native audit trail for backup data access	Cohesity logs backup and restore operations in the Dashboard activity log, but it does not provide a granular audit trail showing who accessed specific backed-up data content. For compliance (HIPAA, PCI-DSS, SOX), combine Cohesity activity logs with PostgreSQL's own pg_audit extension for end-to-end audit coverage.
24	Backups may contain sensitive PII/PHI data	Cohesity backs up the entire data directory including all databases. If some databases contain sensitive PII, PHI, or regulated data, the backup snapshot inherits the same compliance classification. Ensure the Cohesity storage domain is configured with appropriate encryption, access controls, and data retention policies that meet your regulatory requirements.
25	Non-secure gRPC transmits data unencrypted	By default, data transfer between the PostgreSQL host and the Cohesity cluster uses non-secure gRPC (unencrypted). For environments handling sensitive data or subject to network compliance requirements, enable secure gRPC (TLS) even though it increases transfer time. Without TLS, backup data traverses the network in plaintext.
26	No immutable backup lock on PostgreSQL host	While Cohesity provides immutable snapshots on the cluster storage side, the WAL archive files and temporary data on the PostgreSQL host itself are not immutable. A ransomware attack on the PostgreSQL host could corrupt local WAL archives before they are transferred. Ensure host-level security hardening (SELinux, file integrity monitoring, restricted access).

10.8 Scalability and Multi-Instance Limitations

#	Limitation	Detail / Impact / Workaround
27	One source registration per PostgreSQL instance	Each PostgreSQL instance running on a host requires its own separate source registration in Cohesity. If you run multiple PostgreSQL instances (e.g., PG16 on port 5432 and PG17 on port 5433) on the same host, each must be registered independently with unique authentication credentials.
28	No multi-tenancy at the connector level	The Cohesity PostgreSQL Connector does not natively support multi-tenant PostgreSQL deployments where

		different tenants share the same cluster. Backup is always cluster-level, so all tenant data is included in every backup. To isolate tenant backups, separate tenants into distinct PostgreSQL instances.
29	Large database backup window challenges	For very large PostgreSQL databases (multi-TB), full backups can take many hours and consume significant network bandwidth. Plan backup windows carefully and consider: (a) increasing concurrency settings in the Protection Group, (b) using dedicated backup network interfaces, (c) scheduling full backups during maintenance windows, (d) relying on frequent incremental/differential backups to minimize full backup frequency.
30	No backup throttling or bandwidth limiting	Cohesity does not provide native bandwidth throttling for PostgreSQL backups at the connector level. A full backup of a large database will use as much network and I/O bandwidth as available, which can impact production database performance. Schedule large backups during off-peak hours and monitor I/O with tools like iostat and iftop during backup runs.

10.9 Real-World Scenario Examples

The following scenarios illustrate how the above limitations may manifest in real production environments:

Scenario 1: Accidental DROP TABLE on a single database

A developer accidentally runs `DROP TABLE customers` on the 'sales' database. Because Cohesity only supports cluster-level restore, you cannot selectively restore just the 'sales' database or the 'customers' table. You must either restore the entire cluster (affecting all databases) or use a parallel restore to a new instance on port 5433, then manually extract the dropped table using `pg_dump` from the restored instance and `pg_restore` into production.

Scenario 2: Major version upgrade from PostgreSQL 16 to 17

You cannot use Cohesity backups to migrate from PG16 to PG17 because cross-version restore is not supported. The correct approach is: (1) Take a final Cohesity backup of PG16 as a safety net. (2) Use `pg_upgrade` or `pg_dump/pg_restore` to perform the actual version migration. (3) After the upgrade to PG17, re-register the source in Cohesity, create new Protection Policies and Groups for PG17, and run the first full backup.

Scenario 3: HA failover during a backup window

Your Patroni-managed HA cluster experiences a failover from node-A to node-B during a scheduled incremental backup. The in-progress backup on node-A will fail. Once node-B becomes the new primary, the next backup run will automatically be a FULL backup (not incremental), consuming more storage and time. If you registered using node-A's IP, you must update the source registration to point to node-B or, preferably, use a floating VIP that follows the primary.

Scenario 4: PITR after a 2-hour agent downtime

The Cohesity agent was offline for 2 hours (e.g., during an OS patching window). During this time, three scheduled log backups were missed, creating a gap in the WAL backup timeline.

After the agent comes back online, PITR to any timestamp within the 2-hour gap is NOT possible. PITR is only available for timestamps after the next successful full or incremental backup completes. To minimize this risk, schedule agent/OS maintenance outside of backup windows and reduce the log backup interval.

11. Troubleshooting Guide

This section provides structured troubleshooting procedures for the most common issues encountered during Cohesity agent deployment, PostgreSQL source registration, backup execution, and restore operations.

11.1 Agent Installation and Connectivity Issues

Symptom	Root Cause	Resolution
Agent RPM installation fails	Missing dependencies, incompatible OS version, or insufficient disk space (requires 360 MB).	Run: <code>rpm -ivh --test <rpm></code> to see dependency errors. Verify: <code>df -h /opt</code> . Ensure AlmaLinux 8/9 x86_64.
cohesity-agent service fails to start	Port 50051 already in use, SELinux blocking, or corrupted installation.	Check: <code>ss -tlnp grep 50051</code> . Check: <code>journalctl -u cohesity-agent -n 50</code> . Try: <code>setenforce 0</code> (temporary). Reinstall agent if needed.
Agent not reachable from Cohesity cluster	Firewall blocking port 50051, network routing issue, or agent not running.	Verify: <code>firewall-cmd --list-ports</code> . Test: <code>telnet <pg-host> 50051</code> from Cohesity node. Verify: <code>systemctl status cohesity-agent</code> .
cohesityagent user not created	RPM post-install script failed or was skipped.	Manually create: <code>useradd -m cohesityagent</code> . Then restart: <code>systemctl restart cohesity-agent</code> .
Agent version mismatch warning	Agent version not compatible with cluster version.	Download matching agent version from the Cohesity Dashboard for your cluster version. Upgrade: <code>rpm -Uvh <new-rpm></code> .

Diagnostic commands — Agent connectivity:

```
# 1. Verify agent service is running
sudo systemctl status cohesity-agent
sudo systemctl is-enabled cohesity-agent

# 2. Verify port 50051 is listening
sudo ss -tlnp | grep 50051
sudo netstat -tlnp | grep 50051

# 3. Check firewall rules
sudo firewall-cmd --list-all
sudo firewall-cmd --query-port=50051/tcp

# 4. Test connectivity from Cohesity to PG host
# (Run from Cohesity node or another machine)
```

```
telnet <pg-host-ip> 50051
nc -zv <pg-host-ip> 50051

# 5. Check agent logs for errors
journalctl -u cohesity-agent --since '2 hours ago' \
    --no-pager | tail -100
ls -lt /opt/cohesity/agent/logs/ | head -10
cat /opt/cohesity/agent/logs/agent.log | tail -50
```

11.2 Source Registration Failures

Symptom	Root Cause	Resolution
Registration fails with 'Cannot reach host'	Agent not running, wrong IP/hostname, or firewall blocking.	Verify agent is running on port 50051. Confirm the IP entered matches the PostgreSQL host. Check firewall and network path.
Registration fails with 'Script path not found'	PostgreSQL Connector RPM not installed, or wrong path specified.	Verify: ls /opt/cohesity/agent/uda_scripts/. If empty, install the connector RPM. Confirm the exact path used in the UI matches.
Registration fails with authentication error	Wrong PostgreSQL credentials, pg_hba.conf blocking, or user lacks superuser privileges.	Test locally: psql -U cohesity_backup -h 127.0.0.1 -d postgres. Check pg_hba.conf allows md5/scram auth for the backup user. Verify: \du cohesity_backup shows Superuser.
Registration fails with Java error	Java not installed, not in PATH for cohesityagent user, or wrong version.	Test: sudo -u cohesityagent java -version. Must be 1.8+. Add Java to PATH in ~/.bashrc for cohesityagent user.
SSL certificate error during registration	Certificate file not found, wrong permissions, or expired certificate.	Verify cert path and permissions: ls -la /opt/certs/<clustername>.cfg. Ensure cohesityagent user has read access. Regenerate cert if expired.

Diagnostic commands — Registration:

```
# 1. Verify connector scripts exist
ls -la /opt/cohesity/agent/uda_scripts/
ls -la /opt/cohesity/agent/uda_scripts/*.sh

# 2. Verify ownership and permissions
stat /opt/cohesity/agent/uda_scripts/
# Should be owned by cohesityagent:cohesityagent
```

```
# 3. Test PostgreSQL authentication
psql -U cohesity_backup -h 127.0.0.1 -d postgres \
  -c "SELECT version();"

# 4. Verify the backup user has superuser
sudo -u postgres psql -c "\du cohesity_backup"

# 5. Check pg_hba.conf for correct entries
cat /apps/pgsql_data/17/pg_hba.conf | grep -v '^#'

# 6. Verify Java for cohesityagent user
sudo -u cohesityagent which java
sudo -u cohesityagent java -version
sudo -u postgres which java
sudo -u postgres java -version
```

11.3 Backup Failures

Symptom	Root Cause	Resolution
Backup fails with WAL archiving error	archive_mode not enabled, archive_command misconfigured, or archive directory permissions wrong.	SHOW archive_mode; must return 'on'. Verify archive_command with SHOW archive_command;. Check: ls -la /apps/pgsql_data/wal_archive/ — must be owned by postgres with 700 permissions.
Backup times out	Database too large for the configured backup window, network bottleneck, or Cohesity storage full.	Check Cohesity storage: Dashboard > Storage. Increase timeout in Advanced Settings. Check network throughput between PG host and Cohesity. Consider increasing concurrency.
Incremental backup runs as full	First run after registration always runs as full. Also occurs after HA failover or if the backup timeline has a gap.	This is expected behavior. Subsequent runs will be incremental. If recurring, check for gaps in the backup timeline caused by pauses or failures.
Log backup fails with 'No new WAL segments'	No write activity on the database since the last log backup, or archive_timeout = 0 with no transactions.	This may be normal for idle databases. Set archive_timeout = 300 (5 min) to force WAL switch periodically. Or generate activity: SELECT pg_switch_wal();
Backup fails with sudoers/permission error	cohesityagent user missing required sudo privileges, or sudoers file was overwritten by config management.	Run: sudo -u cohesityagent sudo -l. Verify all required commands are listed. Re-add entries via visudo. Ensure config management

		(Chef/Puppet/Ansible) preserves the entries.
Backup fails with disk space error	Insufficient space on the PostgreSQL host for temporary backup staging, or WAL archive directory is full.	Check: <code>df -h /apps/pgsql_data/</code> . Clean old WAL archives if safe. Ensure at least 20% free space on the data partition.

Diagnostic commands — Backup issues:

```
# 1. Verify WAL archiving is working
sudo -u postgres psql -c \
    "SELECT * FROM pg_stat_archiver;"
# Check: last_archived_wal, last_failed_wal,
#         failed_count, last_archive_time

# 2. Check archive_mode and archive_command
sudo -u postgres psql -c "SHOW archive_mode;"
sudo -u postgres psql -c "SHOW archive_command;"
sudo -u postgres psql -c "SHOW data_directory;"
sudo -u postgres psql -c "SHOW wal_level;"

# 3. Manually test WAL archiving
sudo -u postgres psql -c "SELECT pg_switch_wal();"
# Then check if new WAL appears in archive dir:
ls -lt /apps/pgsql_data/wal_archive/ | head -5

# 4. Check disk space
df -h /apps/pgsql_data/
df -h /apps/pgsql_data/wal_archive/
du -sh /apps/pgsql_data/17/

# 5. Check sudoers for cohesityagent
sudo -u cohesityagent sudo -l

# 6. View Cohesity backup logs from Dashboard
# Dashboard > Protection > Click Group >
# Click Run > Click Object > View Logs
```

11.4 Restore Failures

Symptom	Root Cause	Resolution
Restore fails with 'Port already in use'	Target port (e.g., 5432) is occupied by the running original PostgreSQL instance.	Stop the original instance: <code>systemctl stop postgresql-17</code> . Or restore with an alternate port: <code>--target-dir</code> with <code>pg_ctl -o '-p 5433'</code> .

Restore fails with 'Permission denied'	Target restore directory not owned by postgres, or SELinux blocking write.	Pre-create and set ownership: <code>mkdir -p /apps/pgsql_data/17_restored && chown postgres:postgres</code> . Check: <code>getenforce</code> . If Enforcing, run <code>restorecon -R</code> on the target directory.
PITR fails with 'WAL not available'	Requested recovery point falls in a gap in the WAL backup timeline, or log backups were not configured.	Verify log backups were enabled in the Protection Policy. Check the recovery point is within available backup range. Choose a different recovery point after the gap.
Restore completes but database won't start	Corrupted restore data, missing WAL segments for recovery, or mismatched PostgreSQL version.	Check <code>pg_ctl</code> start output and postgresql log. Verify PG version matches. Check: <code>cat /apps/pgsql_data/17_restored/PG_VERSION</code> . Ensure <code>recovery.signal</code> exists for PITR.
Restored database shows old data	Restored from an older snapshot rather than the latest, or PITR timestamp was incorrect.	Verify the selected snapshot timestamp in the Cohesity Dashboard matches expectations. For PITR, confirm the exact timestamp format and timezone.
systemd does not track the restored instance	Cohesity uses <code>pg_ctl</code> directly rather than <code>systemctl</code> to start the database after restore.	Stop with: <code>pg_ctl stop -D /apps/pgsql_data/17</code> . Then start via: <code>systemctl start postgresql-17</code> . This re-attaches the process to systemd.

Diagnostic commands — Restore issues:

```
# 1. Check if target port is in use
sudo ss -tlnp | grep 5432
sudo ss -tlnp | grep 5433

# 2. Verify target directory ownership
ls -la /apps/pgsql_data/17_restored/
stat /apps/pgsql_data/17_restored/

# 3. Check PostgreSQL version of restored data
cat /apps/pgsql_data/17_restored/PG_VERSION

# 4. Try manual start and check errors
sudo -u postgres /usr/pgsql-17/bin/pg_ctl start \
  -D /apps/pgsql_data/17_restored \
  -l /tmp/pg_restore_startup.log
cat /tmp/pg_restore_startup.log
```

```
# 5. Check for recovery.signal (PITR)
ls -la /apps/pgsql_data/17_restored/recovery.signal

# 6. View PostgreSQL startup log
tail -50 /apps/pgsql_data/17_restored/log/\
    postgresql-*.log
```

11.5 SELinux Troubleshooting

SELinux in Enforcing mode can block the Cohesity agent, PostgreSQL Connector scripts, and data transfer operations. The following steps help diagnose and resolve SELinux issues:

```
# 1. Check current SELinux status
sestatus
getenforce

# 2. Temporarily set to Permissive (for testing)
sudo setenforce 0
# Re-run the backup/registration to test
# If it works, SELinux was the issue

# 3. Check SELinux audit logs for denials
sudo ausearch -m AVC -ts recent
sudo ausearch -c 'java' --raw | audit2allow
sudo ausearch -c 'cohesity' --raw | audit2allow

# 4. Create a custom SELinux policy module
sudo ausearch -c 'java' --raw | audit2allow -M \
    cohesity_pg
sudo semodule -i cohesity_pg.pp

# 5. Set correct SELinux contexts
sudo semanage fcontext -a -t postgresql_db_t \
    '/apps/pgsql_data(/.*)?'
sudo restorecon -Rv /apps/pgsql_data/

# 6. Allow Cohesity agent port
sudo semanage port -a -t cohesity_port_t \
    -p tcp 50051

# 7. Re-enable Enforcing after fixing
sudo setenforce 1
```

NOTE: For production environments, always create proper SELinux policy modules rather than permanently disabling SELinux (setenforce 0). Disabling SELinux weakens the overall security posture of the server.

11.6 General Diagnostic Checklist

Run through this checklist when diagnosing any issue:

#	Check	Command
1	Cohesity Agent service running	<code>systemctl status cohesity-agent</code>
2	Agent listening on port 50051	<code>ss -tlnp grep 50051</code>
3	Cohesity RPMs installed	<code>rpm -qa grep cohesity</code>
4	Connector scripts present	<code>ls /opt/cohesity/agent/uda_scripts/*.sh</code>
5	Java accessible by all users	<code>sudo -u cohesityagent java -version</code>
6	PostgreSQL running and accessible	<code>pg_isready -h 127.0.0.1 -p 5432</code>
7	WAL archiving enabled and working	<code>psql -c "SELECT * FROM pg_stat_archiver;"</code>
8	Backup user has superuser role	<code>psql -c "\du cohesity_backup"</code>
9	Sudoers correctly configured	<code>sudo -u cohesityagent sudo -l</code>
10	Firewall ports open	<code>firewall-cmd --list-ports</code>
11	SELinux not blocking operations	<code>getenforce && ausearch -m AVC -ts recent</code>
12	Disk space sufficient	<code>df -h /apps/pgsql_data/</code>
13	Agent logs show no errors	<code>journalctl -u cohesity-agent --since '1hr ago'</code>
14	Network connectivity to Cohesity cluster	<code>curl -sk https://<cluster-vip>/login</code>

11.7 Performance and Timeout Issues

Large databases or constrained networks can cause backup performance degradation and timeouts. Use the following diagnostic approach:

Symptom	Root Cause	Resolution
Full backup exceeds SLA window	Database size too large for available bandwidth or concurrency too low.	Increase concurrency in Protection Group settings. Use a dedicated backup network. Schedule full backups during maintenance windows. Rely on frequent incrementals to reduce full backup frequency.
Backup causes production slowdown	pg_basebackup consuming excessive I/O or CPU on the host.	Monitor with: <code>iostat -x 5</code> , <code>top</code> , and <code>iftop</code> during backups. Consider <code>ionice/nice</code> for the backup process. Schedule during off-peak hours. Check if <code>checkpoint_completion_target</code> is

		tuned (recommended: 0.9).
Network throughput is low during backup	Network congestion, MTU mismatch, or NIC bonding issues between PG host and Cohesity.	Test with: <code>iperf3 -c <cohesity-ip> -t 30</code> . Check MTU: <code>ip link show</code> . Verify NIC bonding: <code>cat /proc/net/bonding/bond0</code> . Consider jumbo frames (MTU 9000) if switches support it.
Cohesity Dashboard shows 'Slow Data Transfer' warning	gRPC channel degraded, TLS overhead, or Cohesity storage domain near capacity.	If using secure gRPC, test without TLS to isolate overhead. Check Cohesity storage: Dashboard > Storage Domains. Verify: no packet loss on network with <code>ping -c 1000 <cohesity-ip></code> .
Incremental backup almost as large as full	High change rate on the database (heavy OLTP workload), bulk loads, or VACUUM FULL running between backups.	This is expected for databases with high change rates. Review change rate with: <code>SELECT pg_database_size(current_database());</code> . Consider more frequent incrementals to spread the delta. Avoid VACUUM FULL during backup windows.

Performance diagnostic commands:

```
# 1. Check database size
sudo -u postgres psql -c \
    "SELECT pg_database.datname, \
    pg_size_pretty(pg_database_size(\
    pg_database.datname)) AS size \
    FROM pg_database ORDER BY \
    pg_database_size(pg_database.datname) DESC;"

# 2. Monitor I/O during backup
iostat -x 5 3
# Look for: %util > 80% (I/O bottleneck)

# 3. Monitor network throughput
# Install iftop: sudo dnf install -y iftop
sudo iftop -i eth0 -n

# 4. Check checkpoint tuning
sudo -u postgres psql -c \
    "SHOW checkpoint_completion_target;"
# Recommended: 0.9

# 5. Test network bandwidth to Cohesity
# Install iperf3: sudo dnf install -y iperf3
```



```
# On Cohesity node: iperf3 -s
# On PG host:
iperf3 -c <cohesity-ip> -t 30 -P 4

# 6. Check WAL generation rate
sudo -u postgres psql -c \
    "SELECT pg_wal_lsn_diff(\
pg_current_wal_lsn(), '0/0') / \
(1024*1024*1024) AS wal_generated_gb;"
```

11.8 WAL Archive Management Issues

Symptom	Root Cause	Resolution
WAL archive directory filling up rapidly	High transaction volume generating WALs faster than they are consumed, or old WALs not being cleaned.	Monitor: <code>du -sh /apps/pgsql_data/wal_archive/</code> . Cohesity manages WAL cleanup after successful backups. If growing fast, increase log backup frequency. For emergency cleanup, remove WALs older than the last successful full backup (verify timestamps first).
WAL archiving falling behind (pg_stat_archiver shows failures)	archive_command failing silently, target directory full, or permissions changed.	Check: <code>SELECT * FROM pg_stat_archiver;</code> — look at failed_count and last_failed_wal. Test the archive_command manually. Verify: <code>df -h /apps/pgsql_data/wal_archive/</code> and <code>ls -la</code> permissions.
postgresql.auto.conf overrides archive_command	Cohesity modifies postgresql.auto.conf during the first backup to set its own archive_command. This overrides the setting in postgresql.conf.	This is expected behavior. After the first Cohesity backup, check: <code>cat /apps/pgsql_data/17/postgresql.auto.conf</code> to see the Cohesity-managed archive_command. Do not manually edit postgresql.auto.conf — Cohesity manages it.
pg_wal directory consuming excessive disk	WAL segments not being archived or recycled. Replication slots preventing WAL cleanup.	Check: <code>SELECT slot_name, active, restart_lsn FROM pg_replication_slots;</code> — inactive slots prevent WAL cleanup. Drop unused slots: <code>SELECT pg_drop_replication_slot('slot_name');</code> . Check: <code>SHOW max_wal_size;</code> and adjust if needed.

WAL management diagnostic commands:

```
# 1. Check WAL archiver status
sudo -u postgres psql -c \
```

```
"SELECT archived_count, failed_count, \
last_archived_wal, last_archived_time, \
last_failed_wal, last_failed_time \
FROM pg_stat_archiver;"
```

2. Check WAL archive directory size

```
du -sh /apps/pgsql_data/wal_archive/
ls -lt /apps/pgsql_data/wal_archive/ | head -10
ls -lt /apps/pgsql_data/wal_archive/ | tail -10
```

3. Check pg_wal directory size

```
du -sh /apps/pgsql_data/17/pg_wal/
```

4. Check replication slots (block WAL cleanup)

```
sudo -u postgres psql -c \
"SELECT slot_name, slot_type, active, \
pg_wal_lsn_diff(pg_current_wal_lsn(), \
restart_lsn) / 1024 / 1024 AS lag_mb \
FROM pg_replication_slots;"
```

5. Check postgresql.auto.conf overrides

```
cat /apps/pgsql_data/17/postgresql.auto.conf
```

6. Manually test archive_command

```
sudo -u postgres psql -c \
"SELECT pg_switch_wal();"
sleep 5
ls -lt /apps/pgsql_data/wal_archive/ | head -3
```

11.9 Multi-Node and HA Cluster Issues

Symptom	Root Cause	Resolution
Backup runs on wrong node after failover	Source registered with static IP of old primary. After failover, the new primary has a different IP.	Register using a VIP (virtual IP) that floats with the Patroni/repmgr primary. Or update the source registration in Cohesity Dashboard after each failover.
Backup fails after Patroni switchover	Patroni changed the primary node. The Cohesity agent on the new primary may not be running or registered.	Ensure Cohesity Linux Agent and PostgreSQL Connector are installed on ALL nodes in the HA cluster. Verify: <code>systemctl status cohesity-agent</code> on each node. Use VIP for registration.
Every backup after failover is a full backup	Expected behavior — differential/incremental tracking	This is by design and cannot be avoided. Plan for additional storage

	resets after failover because the data directory state on the new node is considered different.	consumption after failovers. If failovers are frequent, increase the retention buffer or reduce full backup retention to manage storage.
Standby node cannot be used for backup offloading	Cohesity only supports backing up from the active/primary node. Hot standby backups are not supported.	No workaround within Cohesity. If you need standby-based backups to reduce primary load, use pg_basebackup or pgBackRest separately targeting the standby, and manage those backups independently.

HA cluster diagnostic commands:

```
# 1. Check which node is primary (Patroni)
patronictl list
# Or check pg_is_in_recovery():
sudo -u postgres psql -c \
    "SELECT pg_is_in_recovery();"
# false = primary, true = standby

# 2. Verify agent on all nodes
for NODE in node1 node2 node3; do
    echo "--- $NODE ---"
    ssh $NODE systemctl status cohesity-agent \
        | grep Active
done

# 3. Check VIP is pointing to primary
ping -c 1 <vip-address>
# Compare resolved IP with primary node IP

# 4. Verify registration uses VIP
# Cohesity Dashboard > Sources >
#   Check Hostname/IP matches VIP
```

11.10 Agent Upgrade and Migration Issues

Symptom	Root Cause	Resolution
Backup fails after Cohesity cluster upgrade	Agent version on PG host is now incompatible with the upgraded Cohesity cluster.	Always upgrade the Linux Agent on all registered hosts after a Cohesity cluster upgrade. Download the new agent from the upgraded Dashboard. Upgrade: <code>sudo rpm -Uvh el-cohesity-agent-<new>.rpm</code> . Restart: <code>systemctl</code>

		restart cohesity-agent.
Connector scripts missing after agent upgrade	The PostgreSQL Connector RPM was not upgraded alongside the Linux Agent.	Upgrade the connector separately: <code>sudo rpm -Uvh cohesity-postgres-connector-<new>.rpm</code> . Verify scripts: <code>ls /opt/cohesity/agent/uda_scripts/*.sh</code> .
Registration lost after host IP change	The PostgreSQL host's IP address changed (DHCP, network migration) but the Cohesity source registration still references the old IP.	Edit the source in Cohesity Dashboard: Sources > Actions > Edit > Update Hostname/IP. Or unregister and re-register with the new IP. Use static IPs or DNS hostnames for registered sources.
Backup fails after PostgreSQL minor version update	Minor version updates (e.g., 17.1 to 17.2) normally do not break Cohesity backup, but binary path changes or library updates can occasionally cause issues.	After a PG minor version update: (1) Verify <code>pg_isready</code> works. (2) Verify: <code>sudo -u cohesityagent sudo -l</code> still lists correct paths. (3) Run a manual on-demand backup to validate. (4) Check agent logs for errors.

11.11 Escalation Procedures and Log Collection

If the above troubleshooting steps do not resolve your issue, collect the following information before contacting Cohesity Support:

Log collection commands — Run these and bundle the output for support:

```
# === Collect ALL diagnostic info for support ===

DIAG_DIR=/tmp/cohesity_diag_$(date +%Y%m%d)
mkdir -p $DIAG_DIR

# 1. System information
uname -a > $DIAG_DIR/system_info.txt
cat /etc/os-release >> $DIAG_DIR/system_info.txt
df -h >> $DIAG_DIR/system_info.txt
free -h >> $DIAG_DIR/system_info.txt

# 2. Cohesity agent info
rpm -qa | grep cohesity > $DIAG_DIR/cohesity_rpms.txt
systemctl status cohesity-agent \
    > $DIAG_DIR/agent_status.txt 2>&1
journalctl -u cohesity-agent --since '24 hours ago' \
    --no-pager > $DIAG_DIR/agent_journal.log 2>&1

# 3. Agent log files
```

```
cp -r /opt/cohesity/agent/logs/ \
    $DIAG_DIR/agent_logs/ 2>/dev/null

# 4. PostgreSQL information
sudo -u postgres psql -c "SELECT version();" \
    > $DIAG_DIR/pg_info.txt 2>&1
sudo -u postgres psql -c "SHOW ALL;" \
    > $DIAG_DIR/pg_settings.txt 2>&1
sudo -u postgres psql -c \
    "SELECT * FROM pg_stat_archiver;" \
    > $DIAG_DIR/pg_archiver.txt 2>&1

# 5. Configuration files (sanitized)
cat /apps/pgsql_data/17/postgresql.conf \
    | grep -v '^#' | grep -v '^$' \
    > $DIAG_DIR/postgresql.conf.txt
cat /apps/pgsql_data/17/pg_hba.conf \
    | grep -v '^#' | grep -v '^$' \
    > $DIAG_DIR/pg_hba.conf.txt
cat /apps/pgsql_data/17/postgresql.auto.conf \
    > $DIAG_DIR/postgresql.auto.conf.txt 2>/dev/null

# 6. Connector scripts
ls -la /opt/cohesity/agent/uda_scripts/ \
    > $DIAG_DIR/connector_scripts.txt

# 7. Network and firewall
ss -tlnp > $DIAG_DIR/listening_ports.txt
firewall-cmd --list-all \
    > $DIAG_DIR/firewall.txt 2>&1

# 8. SELinux
sestatus > $DIAG_DIR/selinux.txt 2>&1
ausearch -m AVC -ts recent \
    > $DIAG_DIR/selinux_denials.txt 2>&1

# 9. Sudoers (sanitized)
sudo -u cohesityagent sudo -l \
    > $DIAG_DIR/sudoers_check.txt 2>&1

# 10. Java version
java -version > $DIAG_DIR/java.txt 2>&1
sudo -u cohesityagent java -version \
    >> $DIAG_DIR/java.txt 2>&1
```

```
# Bundle everything
tar czf /tmp/cohesity_diag_$(date +%Y%m%d).tar.gz \
  -C /tmp cohesity_diag_$(date +%Y%m%d)/
echo "Diagnostic bundle created at:"
echo "/tmp/cohesity_diag_$(date +%Y%m%d).tar.gz"
echo "Send this file to Cohesity Support."
```

Cohesity Support escalation checklist:

#	Item to Include	How to Collect
1	Cohesity cluster version and node IPs	Dashboard > Settings > Cluster
2	Cohesity agent version on PG host	rpm -qa grep cohesity
3	PostgreSQL version and data directory path	psql -c "SELECT version();"
4	AlmaLinux version and kernel	cat /etc/os-release && uname -r
5	Protection Group name and Policy name	Dashboard > Protection
6	Exact error message (screenshot or copy)	Dashboard > Protection > Run Logs
7	Timestamp of the failure	Note the exact date/time
8	Diagnostic bundle (.tar.gz)	Run the log collection script above
9	Steps to reproduce (if reproducible)	Document exact sequence of actions
10	Recent changes (upgrades, config changes, failovers)	Document any changes in last 7 days

NOTE: Contact Cohesity Support via the Cohesity Support Portal (<https://www.cohesity.com/support/>). For critical production outages affecting backup SLAs, request Priority 1 (P1) escalation. Ensure the Support Channel is enabled on the Cohesity cluster (Dashboard > Settings > Support Channel) to allow remote troubleshooting.

12. Quick Reference Checklist

#	Task	Status
1	AlmaLinux 8/9 updated, Java 11+ installed	☐ Complete
2	PostgreSQL 17/18 installed and initialized (PGDG)	☐ Complete
3	Firewall port 50051/tcp opened	☐ Complete
4	Cohesity Linux Agent RPM installed, service running	☐ Complete
5	Sudoers configured for cohesityagent	☐ Complete
6	PostgreSQL Connector RPM installed (/opt/cohesity/agent/uda_scripts)	☐ Complete
7	WAL archiving configured (archive_mode=on)	☐ Complete
8	Backup user with superuser privileges created	☐ Complete
9	Source registered in Cohesity Dashboard (UDA)	☐ Complete
10	Protection Policy created (full/diff/log schedules)	☐ Complete
11	Protection Group created and first backup validated	☐ Complete
12	CLI/API automation scripts tested (optional)	☐ Complete

End of Document